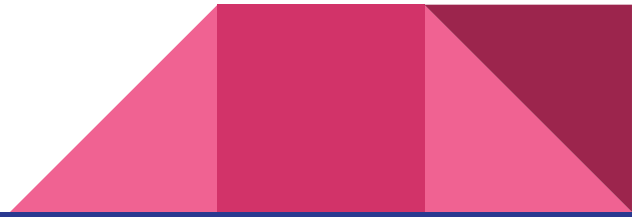
Security Controls in Shared Source Code Repositories

Miguel Fernandez Brazon

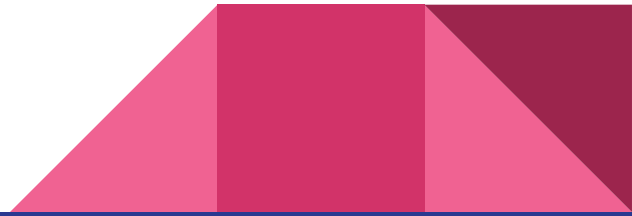
Why it matters

A shared repo often contains credentials, API keys, and configuration that map out your entire system. That makes it a high-value target.



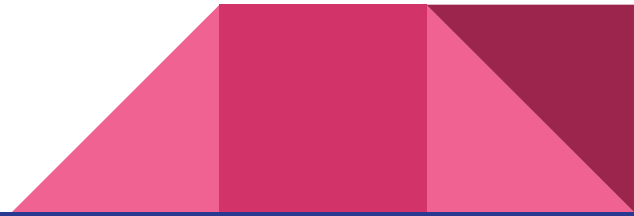
Control who gets in

Give people the minimum access they need to do their job, and audit permissions on a schedule access tends to accumulate as people change teams and no one goes back to trim it. Require multi-factor authentication across the board, no exceptions for admins.



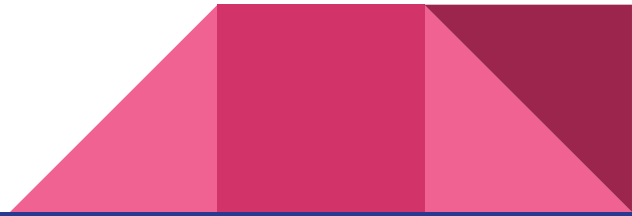
Keep secrets out of the code.

Hard-coded passwords and API keys are a recurring source of breaches. Move them to a secrets manager and reference them at runtime. Use distinct credentials for dev, test, and production too, so a leaked dev key can’t be turned around and used against your live systems.



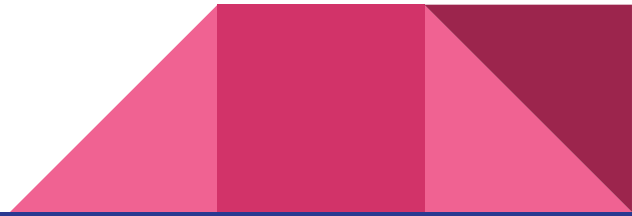
Review code before it ships

Require a second set of eyes on every change before it merges. Turn on branch protection for main so nobody can push directly or approve their own work the review only means something if it can’t be skipped.



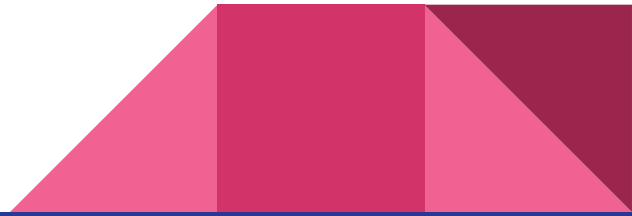
Verify who wrote the code.

Signed commits and make tampering obvious. Paired with mandatory review, they close the gap where someone could forge authorship.



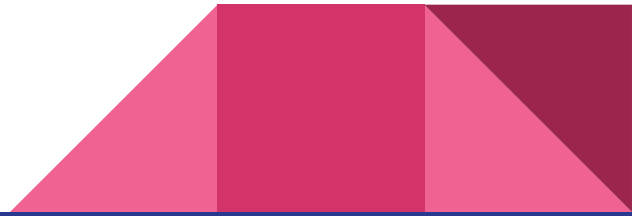
Scan continuously

Outdated dependencies pull vulnerabilities into your project without any visible sign. Run automated scans in your pipeline to flag them before merge, and watch for suspicious third-party packages — typosquatting names and packages that suddenly change maintainers are worth a second look.



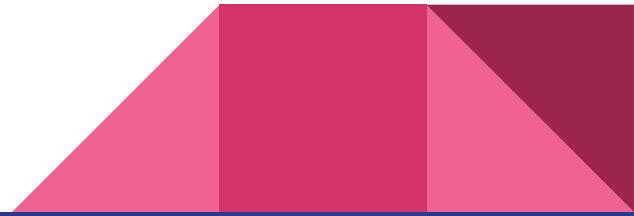
Monitor activity

Log repo access so unusual patterns stand out a login at 3 a.m., a clone from an unfamiliar location, a sudden spike in downloads. Encrypt data in transit so anything intercepted on the way in or out is useless.



The takeaway

No single control catches everything, which is the whole point of layering them. Access control, secrets management, review, and scanning cover for each other’s blind spots. They also only work if they’re built into your normal workflow controls that live outside the day-to-day get bypassed under deadline pressure.



References

https://snyk.io/articles/securing-source-code-repositories/

https://www.harness.io/harness-devops-academy/secure-code-repositories-best-practices

https://checkmarx.com/blog/supply-chain-security/repository-health-monitoring-part-2-essential-practices-for-secure-repositories/

https://orca.security/resources/blog/source-code-security-best-practices/

https://get.assembla.com/blog/source-code-security/